

Teorema CAP și Arhitectura Cloud Computing



Conf.dr. Cristian KEVORCHIAN

Facultatea de Matematică și Informatică

Universitatea din București

ck@fmi.unibuc.ro





Introducere

Cloud Computing sisteme distribuite globale

Probleme fundamentale latență, eșecuri, scalabilitate

Teorema CAP cadru teoretic pentru compromisuri arhitecturale

Importanța algoritmilor de consens în sistemele distribuite

Rolul Azure Service Fabric în arhitectura cloud și antrenare LLM

Teorema CAP

Într-un sistem distribuit, în prezența unei **partiționări a rețelei** (adică pierderea comunicării între noduri), un sistem poate **garanta cel mult două** dintre următoarele trei proprietăți:

1. C – Consistență (en:Consistency):

Toate nodurile văd aceleași date în același timp. Orice citire reflectă cea mai recentă scriere.

2. A – Disponibilitate (en:Availability):

Fiecare cerere primește un răspuns (fie el de succes sau eșec), fără a garanta că este cel mai recent.

3. P – Toleranță la partiționare (en: Partition Tolerance):

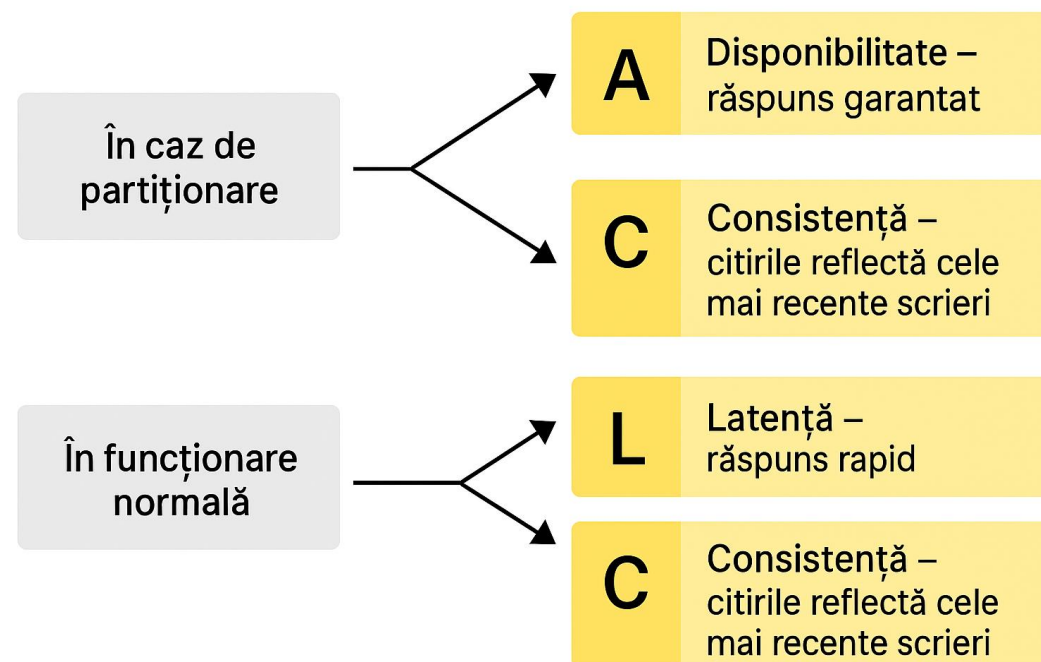
Sistemul continuă să funcționeze chiar dacă unele mesaje sunt pierdute sau nodurile nu pot comunica între ele.

Partiționarea – O Axiomă în Cloud Computing

- Partiționarea este inevitabilă în cloud (**multi-AZ**, **IoT**-instabilitatea rețelei, distribuție geografică, cereri masive, **Edge**-autonomie locata, processare descentralizata, deconectare temporară)
- Arhitectura trebuie să accepte P ca fiind dată
- Alegerea devine: **PC** (consistență) vs. **PA** (disponibilitate)

PACELC – Extensia Teoremei CAP

- **PAC:** În caz de partiție → A vs. C
- **ELC:** În funcționare normală → Latență vs. Consistență
- PACELC = model mai realist pentru sistemele cloud moderne



Tabel PACELC – Stocare Distribuită în Azure

Serviciul Azure	În caz de Partiționare (P)	În funcționare normală (E)	Clasificare PACELC	Observații
Azure Cosmos DB	Configurabil: A sau C	Configurabil: L sau C	Flexibil (PA/EL, PC/EC)	Oferă 5 niveluri de consistență: Strong, Bounded Staleness, Session, Consistent Prefix, Eventual
Azure Table Storage	A – Disponibilitate	L – Latență redusă	PA/EL	Model NoSQL simplu, replicare asincronă, consistență eventuală
Azure Blob Storage	A – Disponibilitate	L – Latență redusă	PA/EL	Ideal pentru fișiere mari, replicare geo-redundantă (GRS)
Azure SQL Database (Hyperscale)	C – Consistență	C – Consistență	PC/EC	Transacții ACID, replicare sincronă între noduri
Azure Data Lake Storage Gen2	A – Disponibilitate	L – Latență redusă	PA/EL	Optimizat pentru big data, bazat pe Blob Storage

Testele de validare practică Jepsen(Kyle Kingsbury)

Definiție: Framework dedicat testării sistemelor distribuite și axat pe identificarea problemelor de consistență și disponibilitate în condiții de eșecuri, partiționări de rețea și comportamente neprevăzute

	Detalii
Metode de testare	Simulări de partiționare, pierderi de pachete, rebooturi, scrieri concurente
Sisteme testate	Cassandra, MongoDB, Redis, Kafka, Etcd, Zookeeper, CockroachDB, TiDB, Consul
Mediu de integrare	Sisteme de baze de date, mesagerie, orchestratoare, cloud, edge computing
Utilizare în DevOps	Integrat în CI/CD pentru testare automată și regresie în sisteme distribuite
Relevanță academică	Utilizat în cercetare pentru validarea algoritmilor de consens și CRDTs (Conflict-free Replicated Data Types)

Algoritmii de consens

- Un **algoritm de consens** este o strategie prin care nodurile unui sistem distribuit ajung la un acord asupra unei valori sau decizii, în ciuda erorilor sau pierderii comunicării între noduri. Alegerea algoritmului este **dictată de ce tip de fault-toleranță dorim să obținem**:
- Dacă dorim **consistență și toleranță la partiționare (CP)**, alegem algoritmi ca **Paxos** sau **Raft**.
- Dacă dorim **availability și partition tolerance (AP)**, se poate merge pe modele "**eventual consistent**", cum ar fi cele din sistemele NoSQL (ex: Dynamo, Cassandra).
- Dacă dorim **consistență și availability** în medii fără partiționări frecvente, putem folosi algoritmi **Byzantine Fault Tolerant (BFT)**, dar cu limitări de scalabilitate.

Algoritmi de Consens în Sisteme Distribuite

	Paxos	Raft	BFT
Poziție CAP	CP	CP	CP (majoritar) sau AP
An apariție	1989/1998	2013	1999 (PBFT)
Complexitate	Foarte ridicată	Medie	Foarte ridicată
Toleranță defectiuni	$f < n/2$	$f < n/2$	$f < n/3$
Overhead mesaje	Moderat	Moderat	Ridicat ($O(n^2)$ sau $O(n^3)$)
Model amenințări	Crash faults	Crash faults	Byzantine faults
Latență consens	2+ round-trips	2 round-trips	3-4 round-trips
Use case principal	Sisteme distribuite clasice	Sisteme cloud-native	Blockchain, sisteme adversariale

Algoritmul de consens = euristică pentru fault-toleranță

Alegerea unui algoritm de consens este o decizie strategică de inginerie care definește contractul de servicii al sistemului (cât de consistent este, cât de des este disponibil) și reflectă în mod direct compromisurile făcute pentru a atinge o anumită formă de fault-toleranță.

SLA-ul este o reflectare a modului în care alegi algoritmul de consens și proiectezi fault tolerance (Consens + Fault Tolerance → SLA).

Algoritmul de consens nu este o soluție universală

- Această idee este fundamentală în sistemele distribuite în general și în cloud computing în particular.
- Nu există un algoritm unic (fie acesta Paxos, Raft sau un alt protocol) care să dețină superioritate absolută. Fiecare algoritm vine cu un pachet de **compromisuri (trade-offs)** în termeni de complexitate, latență și performanță. De exemplu:
 - **Paxos** este mai greu de implementat și de înțeles, dar oferă o garanție teoretică de toleranță la erori superioară.
 - **Raft** a fost conceput pentru a fi mai ușor de înțeles și implementat, sacrificând o mică parte din generalitatea lui Paxos.

Factorii de Adaptare Sunt Cruciali

- Consensul este o **strategie adaptivă** la condițiile specifice ale sistemului:
- **Tipul de Erori Tolerate (Fault Tolerance):**
 - **Crash Failures:** Erori în care un nod pur și simplu se oprește (tolerat de majoritatea algoritmilor de tip Paxos/Raft).
 - **Byzantine Failures (Erori Arbitrare):** Erori în care un nod poate acționa malițios sau poate trimite informații contradictorii altor noduri (necesită algoritmi specifici, cum ar fi **Practical Byzantine Fault Tolerance - pBFT**, folosiți și în unele blockchain-uri-dBFT de la NEO). Majoritatea serviciilor cloud folosesc protocoale mai simple, presupunând medii de încredere (crash failures).
- **Prioritățile Teoremei CAP:**
 - Aceasta este **esența deciziei arhitecturale**. Conform teoremei CAP, un sistem distribuit poate garanta doar două din trei proprietăți: **Consistență**, **Availability** (Disponibilitate) și **Partition Tolerance** (Toleranță la Partajare)(Sharding).
 - Algoritmii de consens puternici (Paxos, Raft) sacrifică adesea **Disponibilitatea** (sistemul se oprește din a servi cereri dacă există o partajare de rețea) pentru a menține **Consistența**.

Mecanismul Quorumului Majoritar ($N/2 + 1$)

Garanția de Consistență a algoritmilor Paxos și Raft se bazează pe regula **cvorumului majoritar**:

- **Cerința de Commit:** O scriere (tranzacție) este considerată finalizată și permanentă (commit) doar după ce a fost înregistrată pe un **cvórum majoritar** de replici ($N/2 + 1$, unde N este numărul total de replici), inclusiv pe replica primară (Leader).
- **Garanția C:** Deoarece o majoritate (mai mult de jumătate din noduri) deține întotdeauna cele mai recente date, este imposibil ca două cvorumuri majoritare să fie de acord asupra unor stări diferite. Acest lucru asigură **Consistența Puternică (Strong Consistency)**, sau *linearizabilitatea*, garantând că fiecare citire primește cel mai recent rezultat al unei scrieri reușite.

Toleranța la Partiție (P)

- Aceeași regulă de cvorum ($N/2 + 1$) permite sistemului să tolereze partiționarea (P) și eșecul nodurilor (până la $N/2$ noduri).
- **Paxos** și **Raft** precum și mecanismele derivate utilizate de sisteme precum **ZooKeeper** (care folosește ZAB, un protocol asemănător Paxos), sunt clasificați fundamental ca sisteme **CP (Consistență și Toleranță la Partiționare)**.
- Această clasificare este obligatorie, deoarece scopul principal al acestor protocoale este de a menține o singură versiune a stării în tot sistemul, indiferent de eșecurile de conectare în rețea

De exemplu:

Într-o familie de 5 replici, se pot pierde 2 noduri, iar sistemul (cu 3 noduri rămase) poate continua să formeze un cvorum majoritar și să proceseze scrieri.

Azure Service Fabric — Sistem de Management CP la Scară Globală

Azure Service Fabric este o platformă de orchestrare distribuită, proiectată pentru a gestiona aplicații scalabile și reziliente pe o infrastructură formată din milioane de servere commodity.

Conform cu teorema CAP, Service Fabric prioritizează:

- **Consistența (C):** prin replicare sincronă și consens (Paxos/Raft), asigură că toate nodurile au o vedere coerentă asupra stării aplicației.
- **Toleranța la partiții (P):** sistemul continuă să funcționeze chiar și în condiții de pierdere parțială a conectivității între noduri.
- **Disponibilitatea (A):** poate fi sacrificată temporar pentru a menține consistența în caz de pierdere de quorum.

Caracteristici cheie

Caracteristică	Descriere
Commodity Servers	Servere standard, fără hardware specializat, distribuite global în centrele de date Azure
Replicație CP	Folosește algoritmi de consens (Paxos/Raft) pentru replici stateful, asigurând consistență puternică
Failover Automat	Detectează defecțiuni și promovează automat o replică secundară la rolul de Primary
Orchestrare Inteligentă	Plasare, monitorizare și actualizare automată a serviciilor în funcție de “sănătatea nodului” și resursele alocate
Scalabilitate Elastică	Se adaptează dinamic la cererea aplicației, fără intervenție manuală
Distribuție Globală	Rulează pe milioane de noduri în centre de date Azure din întreaga lume
Toleranță la Partiții	Continuă să funcționeze în condiții de pierdere parțială a conectivității între noduri
Consistență Prioritară	Menține o vedere coerentă asupra stării aplicației, chiar cu costul temporar al disponibilității

Replicare și Consens în Service Fabric-Replicare

- Service Fabric folosește un mecanism propriu de consens și replicare, **similar conceptual cu ZooKeeper**, dar implementat de o manieră diferită:
- **Componenta principală: Reliable Collections**(structuri de date ca Dictionary sau Queue) + **Replicator**
- **Reliable State Manager**: gestionează starea persistentă a serviciilor stateful.
- **Replicator**: asigură replicarea datelor între noduri.
- **Primary Replica**: coordonează scrierile.
- **Secondary Replicas**: primesc datele prin replicare sincronă sau asincronă.
- **Quorum**: scrierile sunt considerate reușite doar dacă sunt replicate la un număr minim de replici (majoritate).

Replicare și Consens în Service Fabric-Consens

- Nu este ZooKeeper sau Raft, ci un protocol intern Service Fabric bazat pe **quorum și replicare deterministă**.
 - a) Quorumul este numărul minim de replici care trebuie să fie active și sincronizate pentru ca o operație să fie considerată validă.
 - b) În mod tipic, Service Fabric folosește o configurație de replici: 1 primară + N secundare.
 - c) Pentru ca o scriere să fie "committed", trebuie să fie replicată cu succes pe majoritatea replicilor (quorumul).
 - d) Dacă numărul de replici active scade sub acest un prag, se produce o pierdere de quorum (quorum loss), iar partiția devine indisponibilă
 - e) Service Fabric implementează o replicare deterministă pentru a se asigura că toate replicile aplică modificările în aceeași ordine și în același mod.
 - f) Replica primară coordonează toate operațiile de scriere și le transmite replicilor secundare.
 - g) Fiecare replică aplică modificările într-un log de operații, păstrând ordinea exactă.
 - h) Acest model permite failover rapid și sigur: dacă replica primară eșuează, o replică secundară poate fi promovată fără pierderi de date sau inconsistențe
- **Failover automat**: dacă replica primară eșuează, se alege o nouă primară din replicile secundare.
- **Consistency levels**: pot fi configurate (WriteQuorum, ReadQuorum etc.).

Availability Zones (AZ) – Replicare Intra-regională

- **Zonă 1 / Zonă 2 / Zonă 3:** Trei datacenter fizice distincte din aceeași regiune Azure.
- **Noduri distribuite:** Fiecare zonă găzduiește un subset de noduri din același node type.
- **Replici Service Fabric:**
 - **Primary Replica** într-o zonă.
 - **Secondary Replicas** în celelalte două.
- **Quorum:** Se menține consistența chiar dacă o zonă cade.
- **Failover automat:** Dacă zona cu replica primară cade, una dintre secundare este promovată.

Geographical Zones – Replicare Inter-regională

- **Regiunea 1 (ex: West Europe) și Regiunea 2 (ex: North Europe):** Regiuni Azure complet separate geografic.
- **Clustere separate:** Fiecare regiune are propriul cluster Service Fabric.
- **Replicare manuală:**
 - Nu există replicare automată a stării între regiuni.
 - Se pot folosi backup-uri, replicare custom sau sincronizare aplicațională.
- **Failover:** Necesită intervenție manuală sau orchestrare externă (ex: Azure Traffic Manager).
- **Consistență CP:** Dificil de garantat din cauza latenței mari.

Rolul Service Fabric în clusterul OpenAI din Azure(1)

1. Orchestrare de microservicii

- Service Fabric este folosit pentru a gestiona componentele auxiliare ale infrastructurii LLM:
 - **Servicii de monitorizare**
 - **Servicii de control al accesului**
 - **Servicii de routing intern**
 - **Servicii de logging și audit**
- Acestea sunt **stateless** și **stateful**, distribuite pe noduri bare-metal sau virtualizate.

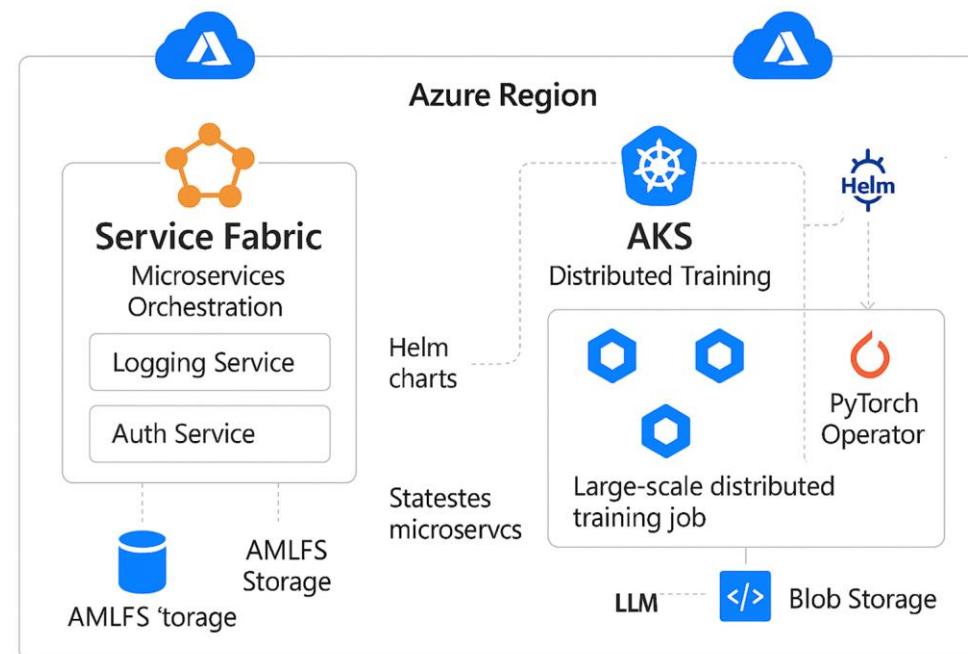
2. Consens și replicare CP

- Pentru serviciile critice (ex: controlul accesului la modele), Service Fabric folosește replicare **stateful** cu **quorum** pentru consistență.
- Protocolul intern de replicare este similar cu ZooKeeper, dar integrat nativ în Service Fabric.

Rolul Service Fabric în clusterul OpenAI din Azure(2)

3. Integrare cu infrastructura bare-metal

- Clusterul OpenAI din Azure folosește **noduri dedicate GPU (ex: A100)**.
- Service Fabric poate fi configurat să ruleze pe aceste noduri pentru a gestiona serviciile auxiliare, dar **antrenarea LLM** se face cu:
 - **AKS (Azure Kubernetes Service)**
 - **PyTorch Operator**
 - **Diagrame Helm**



Tehnologii cheie folosite în clusterul OpenAI

Componentă	Rol principal
Service Fabric	Orchestrare servicii auxiliare, consens CP
AKS + PyTorch Operator	Antrenare distribuită LLM
AMLFS / Blob Storage	Stocare scalabilă pentru date și modele
RDMA Networking	Comunicare rapidă între noduri GPU
Helm Charts	Automatizare și configurare a workload-urilor

- **PyTorch Operator**- controller Kubernetes care automatizează rularea joburilor de antrenare distribuită cu PyTorch.Face parte din ecosistemul Kubeflow având rolul principal de a permite definirea unui job de antrenare în format YAML. Poate gestiona distribuția datelor,sincronizarea între noduri, dar și scalarea automată a workerilor.
- **AMLFS** (Azure Machine Learning File System) este denumirea informală folosită pentru **Azure Managed Lustre**, un sistem de fișiere distribuit de înaltă performanță, optimizat pentru **workload-uri de tip HPC (High Performance Computing)** și **AI**, inclusiv pentru antrenarea modelelor LLM.
- **RDMA** (Remote Direct Memory Access) este o tehnologie de comunicație de rețea care permite unui computer să acceseze direct memoria altui computer fără implicarea procesorului (CPU) sau a sistemului de operare al gazdei aflate remote